

GESTOR INTELIGENTE DE CLIENTES (GIC)

Arquitectura y Desarrollo en Python POO

Proyecto Mòdulo 4

Grupo 1

ELBA NEIRA



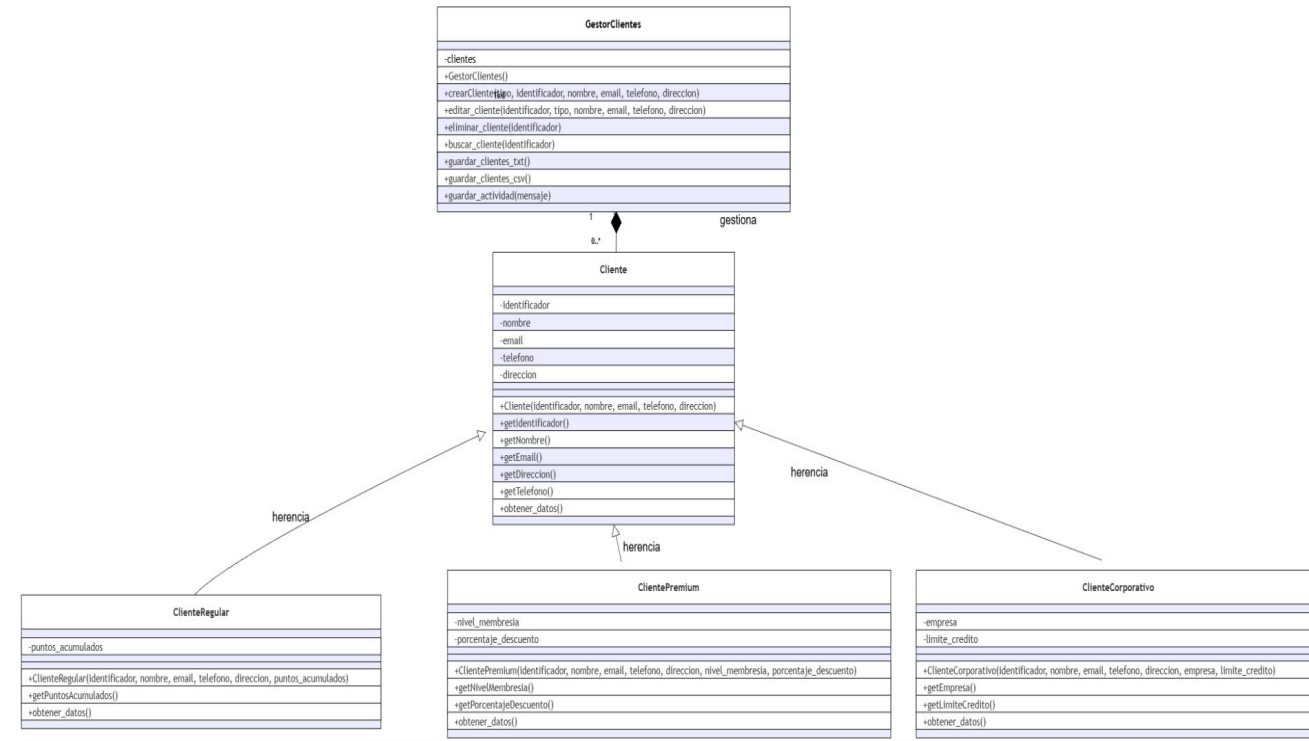
FUNDAMENTOS DEL PROYECTO

Una solución modular diseñada bajo estándares de industria para la gestión eficiente y segura de carteras de clientes.

ARQUITECTURA DEL SISTEMA (UML)

- ✓ **Composición:** El *Gestor de Clientes* posee y controla el ciclo de vida de los objetos (Rombo Negro).
- ✓ **Herencia:** Especialización clara desde una superclase base hacia tipos específicos.
- ✓ **Relación 1 a N:** Un gestor administra múltiples instancias de clientes de forma centralizada.

Gestor de Clientes - UML Proyecto Módulo 4 POO Python



SUPERCLASE Y ENCAPSULAMIENTO

```
def get_identificador(self): return self.__identificador
def get_nombre(self): return self.__nombre
def get_email(self): return self.__email
def get_telefono(self): return self.__telefono
def get_direccion(self): return self.__direccion
```



Atributos Privados

Uso de `__identificador` para prevenir acceso directo externo y garantizar la integridad.



Métodos Getter

Interfaz pública controlada para acceder a los datos sin comprometer el estado del objeto.

HERENCIA Y ESPECIALIZACIÓN



REGULAR

Gestión de puntos acumulados
para fidelización simple.



PREMIUM

Niveles VIP y porcentajes de
descuento dinámicos.



CORPORATIVO

Límites de crédito empresariales y
asociación de marca.

POLIMORFISMO EN ACCIÓN

```
class ClienteRegular(Cliente):  
    def __init__(self, identificador, nombre, email, telefono, direccion, puntos):  
        super().__init__(identificador, nombre, email, telefono, direccion)  
        self.__puntos = puntos  
  
    def get_puntos(self): return self.__puntos  
    def obtener_datos(self):  
        return f"{super().obtener_datos()} | Puntos: {self.__puntos}"
```

Método obtener_datos()

Permite tratar a cualquier objeto como un "Cliente", pero cada uno responde con su propia lógica interna.

Ventaja: El código del gestor no necesita saber qué tipo de cliente es para mostrar su información.



Lógica de Negocio y Robustez del Sistema

GESTIÓN DE EXCEPCIONES

```
def crear_cliente(self, cliente):  
    if self.buscar_cliente(cliente.get_identificador()):  
        raise ValueError("El cliente ya existe")  
    self.clientes.append(cliente)  
    self.registrar_log(f"Agregado cliente ID {cliente.get_identificador()}")
```

- ✓ **Validación Preventiva:** Uso de raise ValueError para campos vacíos.
- ✓ **Formato Estricto:** Verificación de emails con presencia obligatoria de "@".
- ✓ **Integridad CRUD:** Control de duplicados antes de permitir el ingreso a la lista maestra.

PERSISTENCIA Y EXPORTACIÓN



JSON

Estructura jerárquica para intercambio de datos ligero.



CSV

Formato tabular compatible con hojas de cálculo.



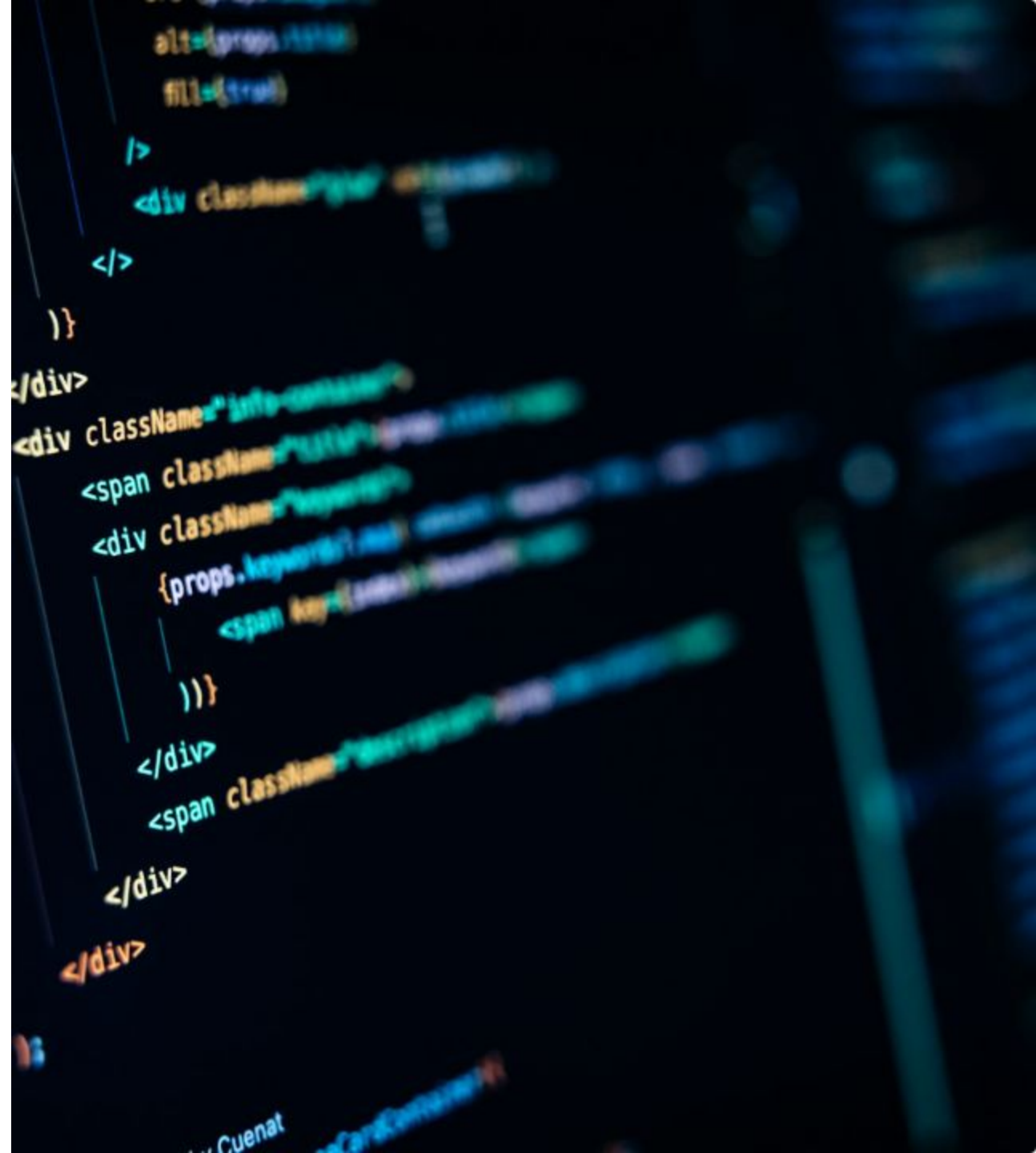
AUDITORÍA

Registro histórico de actividad en archivos .txt

TRAZABILIDAD TOTAL

Cada vez que el sistema realiza una acción crítica, se genera un registro en `actividad.txt`.

Se incluye: **Marca de tiempo**, **Tipo de Acción** e **Identificador** involucrado.





//

"La POO no es solo código; es modelar la realidad para crear sistemas escalables y seguros."

ELBA NEIRA - 2026



github.com/elba-neira/gic-python

IMAGE SOURCES



<https://www.conceptdraw.com/How-To-Guide/picture/uml-apartment-plan.png>

Source: www.conceptdraw.com



https://lookaside.fbssbx.com/lookaside/crawler/media/?media_id=122139177951017180

Source: www.facebook.com



<https://assets.jimmysong.io/images/blog/json-prompt-3d-icon/3d-images.webp>

Source: jimmysong.io



https://png.pngtree.com/png-vector/20260323/ourlarge/pngtree-set-of-document-icons-in-various-styles-vector-png-image_18644880.webp

Source: pngtree.com



<https://images.unsplash.com/photo-1699885960867-56d5f5262d38?fm=jpg&q=60&w=3000&auto=format&fit=crop&ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1yZWxhdG>

[VkfDE0fHx8ZW58MHx8fHx8](https://images.unsplash.com/photo-1699885960867-56d5f5262d38?fm=jpg&q=60&w=3000&auto=format&fit=crop&ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1yZWxhdG)

Source: unsplash.com